इलेक्ट्रॉनिकी एवं सूचना प्रौद्योगिकी मंत्रालय
MINISTRY OF ELECTRONICS AND INFORMATION TECHNOLOGY

रा.इ.सू.प्रौ.सं NIELIT

SoCTeamup

Sub-module 3

UART Serial Protocol

A Brief Description of UART

From start bits to baud rate — everything you need to know

What We'll Cover

01 What is UART?

02 How UART Works?

03 UART Frame Structure

04 UART TX FSM (rtl/uart_tx.v)

05 UART RX FSM (rtl/uart_rx.v)

06 Baud Rate & Project Configuration

07 Lab Command

08 Waveform Debug

09 Real-World Applications

What is UART?

UART (Universal Asynchronous Receiver-Transmitter) is one of the oldest and most widely used serial communication protocols. It transmits data bit-by-bit over a single wire per direction, without a shared clock signal — making it simple, low-cost, and universally supported.

Developed By	Signal Type	Data Format	Use Case
Originally from the 1960s Popularised with RS-232 standard Standardised across all MCUs	Asynchronous — no clock line TX & RX lines only Full-duplex capable	Start bit + 5-9 data bits Optional parity bit 1 or 2 stop bits	Debug consoles & logging GPS, GSM, BT modules MCU-to-MCU links

How UART Works

UART uses two wires — TX (transmit) and RX (receive). Each side sends and receives independently, with no shared clock. Both devices must agree on the same baud rate.

1 Idle State	The TX line sits HIGH (logic 1) when no data is being sent. Both devices wait at idle — this is the resting voltage.
2 Start Bit	Sender pulls TX LOW (logic 0) for exactly one bit period. This signals the receiver: 'data is coming — start your clock!'
3 Data Bits	5–9 data bits are sent LSB-first. Each bit lasts exactly 1/baud-rate seconds. Receiver samples mid-bit for accuracy.
4 Parity Bit (optional)	An optional error-detection bit. Even parity: total 1s are even. Odd parity: total 1s are odd. Can be omitted (None).
5 Stop Bit(s)	TX returns HIGH for 1 or 2 bit periods. Signals end of frame and gives receiver time to prepare for the next byte.

UART Frame Structure

A single UART frame (byte transmission) consists of: Start → Data → Parity → Stop

Start Bit

Always logic LOW. Triggers receiver to begin sampling. Duration = 1 / baud rate.

Data Bits (5–9)

Payload bits sent LSB first. Standard UART uses 8 bits (one full byte). Receiver samples mid-bit.

Parity Bit

Optional single-bit error check. Even, Odd, or None (most common). Cannot correct errors, only detect.

Stop Bit(s)

1 or 2 bits at logic HIGH. Marks end of frame. Gives receiver time to reset before next start bit.

```
Example:  'H' = 0x48 = 0100_1000  →  TX: 0  0 0 0 1 0 0 1 0  1   (Start | D0→D7 LSB-first | Stop)
```

UART TX FSM — rtl/uart_tx.v

IDLE 2'b00	START 2'b01	DATA 2'b10	STOP 2'b11
tx = 1 (HIGH) Wait for valid=1 Load shift_reg	tx = 0 (start bit) Count 434 cycles then → DATA	tx = shift_reg[0] Shift right each bit bit_cnt 0→7 (×8)	tx = 1 (stop bit) Count 434 cycles then → IDLE

← STOP → IDLE (after 434 cycles)

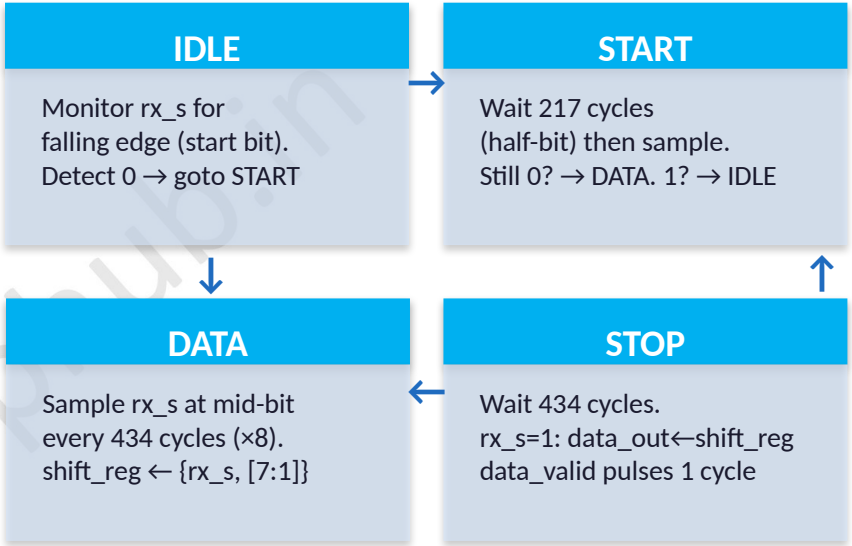
```
Ports:
  clk, reset          — system clock & active-high reset
  data[7:0]           — byte to transmit
  valid               — pulse: assert HIGH for 1 cycle when data is ready
  ready               — HIGH when TX is in IDLE (accepting new byte)
  tx                  — serial output line (connect to RX pin of receiver)

Key Registers:
  shift_reg[7:0]      — loaded from data; right-shifted each bit-period; bit[0] → tx
  clk_cnt[9:0]        — baud counter: counts 0 → 433 (= CLKS_PER_BIT − 1)
  bit_cnt[2:0]        — data bit counter: counts 0 → 7 in DATA state
```

UART RX FSM — rtl/uart_rx.v

2-FF Synchronizer — Why It Exists

External rx arrives asynchronously — its edges may not align to our 50 MHz clock.
Capturing such a signal directly risks metastability: the flip-flop may output
an undefined voltage for an unknown duration.

Fix: 2-stage flip-flop chain:
rx → rx_sync0 → rx_sync1 → rx_s (safe, clock-domain-crossed)




```
Output:  data_out[7:0]   – received byte (valid on data_valid pulse)
         data_valid       – single-cycle HIGH pulse per successfully received byte

Mid-bit timing:  CLKS_HALF_BIT = CLKS_PER_BIT / 2 = 434 / 2 = 217
  → RX waits 217 cycles after detecting start bit before first sample
  → Subsequent bits sampled every 434 cycles (mid-bit of each DATA bit)
```

Baud Rate & Project Configuration

50 MHz Clock · 115200 Baud · CLKS_PER_BIT = 434

Project Calculation

```
CLK_FREQ    =  50,000,000 Hz  (50 MHz)
BAUD_RATE   =     115,200 baud
──────────────────────────────────────
CLKS_PER_BIT  =  50,000,000 ÷ 115,200
              =  434 clock cycles / bit

CLKS_HALF_BIT =  434 ÷ 2  =  217
1 bit period  =  434 × 20 ns  =  8.68 µs
1 byte frame  =  10 bits × 8.68 µs  =  86.8 µs
```

In RTL — uart_tx.v / uart_rx.v

```
localparam CLK_FREQ    = 50_000_000;
localparam BAUD_RATE   = 115_200;
localparam CLKS_PER_BIT = CLK_FREQ / BAUD_RATE; // 434

// In DATA state:
if (clk_cnt == CLKS_PER_BIT - 1) begin
    clk_cnt <= 0;
    shift_reg <= shift_reg >> 1;  // LSB first
    bit_cnt   <= bit_cnt + 1;
end
```

8N1 Configuration means:

8 — Data Bits

uart_tx.v counts bit_cnt 0→7 (8 states in DATA)

N — No Parity

Parity bit position is skipped entirely — saves 1 bit time

1 — Stop Bit

TX returns HIGH for 1 full bit period before next frame

Common Baud Rates (all with 50 MHz clock):

9600	19200	38400	57600	**115200**	230400	921600
÷5208	÷2604	÷1302	÷868	÷434	÷217	÷54

Lab Execution — make uart

Loopback Verification · Verilator Simulation

```
$ make uart
```

1. Compile

Verilator builds:
uart_tx.v + uart_rx.v
uart_top.v + tb_uart_top.v

2. Loopback

uart_top.tx wire-connected
to uart_top.rx internally
(no physical hardware needed)

3. Run TB

Testbench sends bytes:
"hello deepak"
RX captures & compares

4. VCD Out

Produces tb_uart.vcd
(~2.9 MB waveform file)
Open in GTKWave

Expected Terminal Output

```
[UART TB] Sending byte: 0x68 ('h')
[UART TB] Received: 0x68 ('h') ← PASS
[UART TB] Sending byte: 0x65 ('e')
[UART TB] Received: 0x65 ('e') ← PASS
...
[UART TB] String "hello deepak" verified: ALL PASS
```

GTKWave Signals to Add

```
clk                   — 50 MHz system clock
reset                 — active-high reset
uart_top.tx           — serial output (8N1 frame)
uart_top.tx_state[1:0] — 0=IDLE 1=START 2=DATA 3=STOP
uart_tx.clk_cnt[9:0]  — baud counter (0 → 433)
uart_tx.bit_cnt[2:0]  — bit counter (0 → 7)
uart_rx.data_valid    — 1-cycle pulse per received byte
uart_rx.data_out[7:0] — received byte value
```

Debug: data_valid never fires → check reset clears rx_sync to 1'b1 | Wrong byte → verify CLKS_PER_BIT matches TX & RX | Framing error → check CLK_FREQ param

Waveform: Decoding a UART Frame in GTKWave

Transmitting 'H' = 0x48 = 0100_1000

Bit	Cycles	Time (ns)	TX Value	Meaning	Bit Value
START	0–433	0–8,680	tx = 0	Start bit (LOW)	—
D0	434–867	8,680–17,360	tx = 0	D0 = 0 (LSB of 0x48)	0
D1	868–1301	17,360–26,040	tx = 0	D1 = 0	0
D2	1302–1735	26,040–34,720	tx = 0	D2 = 0	0
D3	1736–2169	34,720–43,400	tx = 1	D3 = 1	1
D4	2170–2603	43,400–52,060	tx = 0	D4 = 0	0
D5	2604–3037	52,060–60,740	tx = 0	D5 = 0	0
D6	3038–3471	60,740–69,420	tx = 1	D6 = 1	1
D7	3472–3905	69,420–78,100	tx = 0	D7 = 0 (MSB of 0x48)	0
STOP	3906–4339	78,100–86,800	tx = 1	Stop bit (HIGH)	—

Total: 4340 cycles × 20 ns = 86.8 µs | Read D0→D7 LSB-first: 0 0 0 1 0 0 1 0 → reverse = 0100_1000 = 0x48 = 'H' ✓

Real-World Applications of UART

FT232 / CP2102 USB-UART Bridges

Convert USB to UART for connecting microcontrollers and FPGAs to a host PC serial terminal (PuTTY / screen). The exact interface this project's uart_tx.v would drive.

FPGA Bring-Up & Debug

UART is the first peripheral added to any new SoC or FPGA design — its simplicity makes it the perfect early-stage debug channel. If UART works, the clock tree and basic RTL are correct.

Arduino / STM32 Embedded MCUs

The Serial.print() call on Arduino uses a UART transmitter identical in structure to uart_tx.v — same 8N1 format, same baud rate configuration, same start/stop framing.

THIS Project: PicoRV32 SoC

uart_tx_pin in GTKWave shows the live 8N1 serial stream. tb_top.v UART monitor decodes bytes and prints:
[TB] RX byte: H ← verified ✓
10× 'Hello Deepak from Nielit!' → TEST PASSED

Key Takeaways

UART Protocol	Asynchronous serial, 2 wires (TX + RX), no clock line — both sides must agree on baud rate
8N1 Format	1 start bit (LOW) + 8 data bits (LSB first) + 1 stop bit (HIGH) — no parity in this project
Baud Calculation	CLKS_PER_BIT = 50,000,000 ÷ 115,200 = 434 clock cycles per bit at 50 MHz system clock
TX FSM	rtl/uart_tx.v — 4 states: IDLE / START / DATA / STOP (bit_cnt 0→7 in DATA state)
RX FSM	rtl/uart_rx.v — mid-bit sampling + 2-FF synchronizer (rx_sync0→rx_sync1) for metastability protection
Lab Command	make uart → loopback → tx_byte == rx_byte → PASS → inspect tb_uart.vcd in GTKWave
Full SoC	UART enabled via AXI write to 0x1000_0000 — uart_axi.v wraps TX+RX with AXI-Lite backpressure